

Redis Lua 与 Stream 秒杀异步下单知识点

标记：相关知识

今日学习总结

今天主要学习了三块内容：

1. Lua 脚本的简单使用
2. Redis Stream 消息队列
3. 内部类多线程处理异步任务

其中重点是在高并发秒杀场景中，使用 Redis + Lua + Stream 消息队列，把“资格判断”和“订单创建”拆开处理。

一、为什么使用 Lua 脚本

在秒杀场景中，需要同时判断：

1. 库存是否充足
2. 用户是否重复下单
3. 扣减库存
4. 记录用户下单信息
5. 发送订单消息

这些操作如果分成多条 Redis 命令执行，可能会出现并发安全问题。

Lua 脚本的作用是：把多条 Redis 操作放在一个脚本里执行，Redis 会保证脚本执行过程的原子性。

也就是说，在 Redis 中执行 Lua 脚本时，不会被其他请求插入打断。

二、Redis Stream 的作用

Redis 快速判断用户是否有购买资格后，不直接在当前线程里创建订单，而是把订单信息写入 Redis Stream。

然后后台线程从 Stream 中读取消息，异步创建订单。

整体流程是：

```
1  用户请求
2  ↓
3  Lua 脚本判断库存和一人一单
4  ↓
5  判断成功后，把订单消息写入 Redis Stream
6  ↓
7  后台线程读取 Stream 消息
8  ↓
9  创建订单
10 ↓
11 ACK 确认消息处理完成
```

这样可以降低接口响应时间，也能缓解高并发下数据库的压力。

三、创建消费者组

使用 Redis Stream 时，需要先创建消费者组：

```
1  stringRedisTemplate.opsForStream().createGroup(
2      queueName,
3      ReadOffset.from("0"),
4      "g1"
5  );
```

含义是：

```
1  queueName: Stream 队列名称
2  ReadOffset.from("0")：从队列起始位置开始消费
3  g1: 消费者组名称
```

对应 Redis 命令是：

```
1  XGROUP CREATE stream.orders g1 0 MKSTREAM
```

其中：

```
1  stream.orders: Stream 队列名
2  g1: 消费者组名
3  0: 从头开始读
4  MKSTREAM: 如果 Stream 不存在，就自动创建
```

四、消费者读取消息

消费者读取消息时，需要指定：

1. 消费者组
2. 消费者名称
3. 每次读取数量
4. 阻塞等待时间
5. 绑定的 Stream 队列
6. 读取位置

代码示例：

```
1 List<MapRecord<String, Object, Object>> read =
2     stringRedisTemplate.opsForStream().read(
3         Consumer.from("g1", "c1"),
4         StreamReadOptions.empty()
5             .count(1)
6             .block(Duration.ofSeconds(2)),
7         StreamOffset.create(queueName, ReadOffset.lastConsumed())
8     );
```

含义是：

```
1 Consumer.from("g1", "c1")：消费者组 g1 中的消费者 c1
2 count(1)：每次最多读取 1 条消息
3 block(Duration.ofSeconds(2))：没有消息时最多阻塞 2 秒
4 ReadOffset.lastConsumed()：读取当前消费者组中还没有被消费的新消息
```

对应 Redis 命令是：

```
1 XREADGROUP GROUP g1 c1 COUNT 1 BLOCK 2000 STREAMS stream.orders >
```

其中 > 表示只读取从未被消费者组消费过的新消息。

五、ACK 确认机制

消费者处理完消息后，需要手动 ACK：

```
1 stringRedisTemplate.opsForStream().acknowledge(
2     queueName,
3     "g1",
4     entries.getId()
5 );
```

对应 Redis 命令：

```
1 | XACK stream.orders g1 <message-id>
```

ACK 的作用是告诉 Redis：

```
1 | 这条消息已经被成功处理，可以从 pending-list 中移除。
```

如果不 ACK，这条消息会一直留在 pending-list 中。

六、pending-list 的作用

如果消费者读取了消息，但是处理过程中程序宕机、异常退出，消息没有 ACK。

这条消息不会丢失，而是进入 pending-list。

后续可以通过 `handlePendingList` 再次处理这些未确认消息。

常用命令：

```
1 | XPENDING stream.orders g1 - + 10
```

表示查看消费者组 `g1` 中还没有被 ACK 的前 10 条消息。

处理 pending-list 时，读取位置一般不是 `>`，而是从 0 开始读 pending 消息。

核心思想是：

- 1 | 正常消费：读取新消息
- 2 | 异常恢复：读取 pending-list 中未确认的旧消息

七、相关 Redis 命令整理

```
1 | # 创建消费者组，如果 stream 不存在则自动创建
2 | XGROUP CREATE stream.orders g1 0 MKSTREAM
3 |
4 | # 消费者组读取新消息
5 | XREADGROUP GROUP g1 c1 COUNT 1 BLOCK 2000 STREAMS stream.orders >
6 |
7 | # 查看 pending-list 中未确认的消息
8 | XPENDING stream.orders g1 - + 10
9 |
10 | # 确认消息处理完成
11 | XACK stream.orders g1 <message-id>
12 |
13 | # 查看消费者组信息
14 | XINFO GROUPS stream.orders
```

```
15  
16 # 查看 Stream 队列信息  
17 XINFO STREAM stream.orders
```

八、整体理解

这套方案本质上是：

```
1 Redis Lua 负责快速判断购买资格  
2 Redis Stream 负责异步传递订单消息  
3 后台线程负责真正创建订单  
4 ACK + pending-list 负责保证消息不丢失
```

在高并发场景下，这样做的好处是：

1. Lua 脚本保证 Redis 判断逻辑的原子性
2. Redis 承担高并发的快速判断压力
3. 数据库写入被异步化，避免瞬间打垮数据库
4. Stream 的 pending-list 可以防止消息处理失败后丢失

可以把今天的重点总结成一句话：

秒杀请求先通过 Lua 在 Redis 中完成库存和一人一单判断，成功后把订单消息写入 Redis Stream，再由后台线程异步消费消息创建订单，并通过 ACK 和 pending-list 保证消息可靠处理。